

LWCPU4

MICROCOMPUTERS

MICROCOMPUTERS

MICROCOMPUTERS

MICROCOMPUTERS

HARDWARE & PROGRAMMING MANUAL

8-bit Custom CPU | Logic World Implementation
Revision 6 | 2026

TABLE OF CONTENTS

1. Overview
2. Architecture
3. Instruction Set
4. Assembly Language
5. Memory Map
6. Software & Tools
7. Programming Guide
8. Examples
9. Technical Reference

1. OVERVIEW

1.1 Project Introduction

LWCPU4 is a custom 8-bit CPU designed and implemented using the Logic World game. It features a von Neumann architecture with segmented memory, accumulator-based instruction set, and integrated I/O capabilities.

Project Details:

Property	Value
Type	8-bit Accumulator CPU
Designer	Custom design in Logic World
Word Size	8 bits
Address Space	256 bytes per segment × 256 segments
Assembly	CustomASM syntax

1.2 Key Features

- Two general-purpose registers (A, B)
- Status register with carry, zero, and negative flags
- Segmented memory architecture (Code Segment, Data Segment)
- Conditional jump instructions
- Full arithmetic operations (add, subtract, compare)
- Logical operations (AND, OR, XOR)
- Shift and rotate operations

1.3 Development Tools

Assembler: CustomASM (customasm.exe)

Emulators: Web-based (index.html), CLI (emulators/cli.js)

Hardware Design: Logic World

2. ARCHITECTURE

2.1 CPU Registers

Register	Name	Description
A	Accumulator	Primary register for arithmetic and logic operations
B	Secondary	Secondary register for operations and data
ST	Status	Flags register (C, Z, N)
PC	Program Counter	Current instruction address
CS	Code Segment	Current code segment
DS	Data Segment	Current data segment

2.2 Status Register (ST)

Bit	Flag	Name	Description
0	C	Carry	Set when arithmetic operation generates carry
1	Z	Zero	Set when result is zero
2	N	Negative	Set when result is negative (bit 7 set)

2.3 Register Encoding

Register	Binary	Hex
A	0b01	0x1
B	0b10	0x2
ST	0b11	0x3

2.4 Special Addresses

Address	Use
0x00	Bootloader code
0x10	User program start

3. INSTRUCTION SET

3.1 Instruction Encoding

All instructions are 1-3 bytes. The first byte contains:

Byte 0: [opcode (4 bits) | r1 (2 bits) | r2 (2 bits)]

High nibble (bits 7-4): OpCode

Low nibble (bits 3-0): Register fields

- Bits 3-2: Register 1 (r1)
- Bits 1-0: Register 2 (r2)

3.2 OpCode Summary

Mnemonic	Type	Bytes	Description
SPC	Special	1	Set PC to register
JMP/JIF	Jump	2	Conditional jump
SCD	Control	1	Set condition
MOV	Move/Load/Store	1-3	Register, memory, and segment moves
SEG	Segment	2	Segment operations
ADD	Arithmetic	2-3	Addition
ADC	Arithmetic	2-3	Add with carry
SUB	Arithmetic	2-3	Subtraction
SBC	Arithmetic	2-3	Subtract with carry
CMP	Compare	2-3	Compare (flags only)
AND	Logic	2-3	Logical AND
OR	Logic	2-3	Logical OR
XOR	Logic	2-3	Logical XOR
SHR/ROR/ASR	Shift	1	Shift/Rotate

3.3 Detailed Instructions

SPC (Set PC to Register)

Syntax: `spc reg`

Example: `spc a`

Copies the program counter value into the specified register.

JMP / JIF (Jump)

Unconditional:

Syntax: `jmp addr`

Example: `jmp 0x20`

Register:

Syntax: `jmp reg`

Example: `jmp a`

Conditional:

Syntax: `jif addr`

Example: `jif 0x20`

JMP - Unconditional jump to address. JIF - Jump if condition (set by SCD) is true.

SCD (Set Condition)

Syntax: `scd cond`

Example: `scd z`

Sets the condition flags for conditional jumps.

Condition	Code	Description
c	0b0001	Carry set
z	0b0010	Zero set
n	0b0100	Negative set
nc	0b1001	Not carry
nz	0b1010	Not zero
nn	0b1100	Not negative

MOV (Move)

The MOV instruction handles all register moves, memory loads, memory stores, and segment operations.

Immediate

Syntax: `mov reg, data`

Example: `mov a, 10`

Register to Register

Syntax: `mov reg1, reg2`

Example: `mov a, b`

Load from Memory (Address)

Syntax: `mov reg, [addr]`

Example: `mov a, [0x10]`

Load from Memory (Register)

Syntax: `mov reg, [reg]`

Example: `mov a, [b]`

Store to Memory (Address)

Syntax: `mov [addr], reg`

Example: mov [0x10], a

Store to Memory (Register)

Syntax: mov [reg1], reg2

Example: mov [b], a

Store Immediate to Memory

Syntax: mov [reg], data

Example: mov [b], 10

Set Segment from Immediate

Syntax: mov seg, num

Example: mov cs, 0x10

Set Segment from Register

Syntax: mov seg, reg

Example: mov cs, a

Memory loads update ST flags (Z, N). Segment values: cs = Code Segment, ds = Data Segment.

ADD (Addition)

Syntax: add reg1, reg2

Example: add a, b

Syntax: add reg, data

Example: add a, 10

Adds two values, stores result in first register. Updates C, Z, N flags.

ADC (Add with Carry)

Syntax: adc reg1, reg2

Syntax: adc reg, data

Adds with carry flag included.

SUB (Subtract)

Syntax: sub reg1, reg2

Example: sub a, b

Syntax: sub reg, data

Example: sub a, 10

Subtracts (first - second). Updates flags.

SBC (Subtract with Carry)

Syntax: sbc reg1, reg2

Syntax: sbc reg, data

Subtracts with borrow (carry flag).

CMP (Compare)

Syntax: cmp reg1, reg2

Syntax: cmp reg, data

Non-storing subtraction. Sets C, Z, N flags based on result.

AND (Logical AND)

Syntax: and reg1, reg2

Syntax: and reg, data

OR (Logical OR)

Syntax: or reg1, reg2

Syntax: or reg, data

XOR (Logical XOR)

Syntax: xor reg1, reg2

Syntax: xor reg, data

SHR / ROR / ASR (Shift/Rotate)

Syntax: shr reg

Syntax: ror reg

Syntax: asr reg

Shift right, rotate right, or arithmetic shift right on a register.

4. PROGRAMMING GUIDE

4.1 Hello World

```
#bankdef lc4 {  
    #bits 8  
    #addr 0x00  
    #outp 0  
}  
  
#addr 0x10  
init:  
    mov b, message  
print:  
    mov ds, 0x00  
    mov a, [b]  
    scd z  
    jif .end  
    mov ds, 0x40  
    mov [0], a  
    add b, 1  
    jmp print  
.end:  
    jmp $  
  
message:  
    #d "Hello, World!\n\0"
```

4.2 Memory Operations

Load from memory: Use MOV with address in brackets
`mov a, [0x10]` ; Load from address 0x10

Store to memory: Use MOV with destination in brackets
`mov [0x20], a` ; Store A to address 0x20

5. MEMORY MAP

5.1 Segment Overview

Segment	Use
0x00-0xFF	User defined

The entire segment address space from 0x00 to 0xFF is available for user-defined purposes. Programs, data, and peripherals can be mapped to any segment as needed.

6. SOFTWARE & TOOLS

6.1 Assembler (CustomASM)

```
customasm.exe input.asm -o output.bin
```

6.2 CLI Emulator

```
node emulators/cli.js boot.bin program.bin [-l address] [-b]
```

Options:

-l, --load-addr Load address (default: 0x10)
-b, --bootloader Load bootloader

Controls:

SPACE Run/Pause
R Reset
T Toggle speed
Q Quit

6.3 Web Emulator

Open `index.html` in a web browser to use the visual emulator.

7. TECHNICAL REFERENCE

7.1 Condition Codes

Code	Binary	Name	Meaning
0x1	0001	c	Carry set
0x2	0010	z	Zero
0x4	0100	n	Negative
0x9	1001	nc	Not carry
0xA	1010	nz	Not zero
0xC	1100	nn	Not negative